INTE 201: PROGRAMMING 3 (Structured Programming)

Name: Gueta, Ethan Ulrick D.
Instructor: Sir Victor James Ordoñez
Year and Section: DIT 2-3
Date Passed: October 25, 2026

Activities – Week 4 – Control Flow II – Looping & Jump
for • while • do-while • Nested Loops • break • continue • return

BASIC

Number & Square Printer

Use a for loop to print a sequence of numbers alongside their squares.

Requirements:

- Use a for loop to count from 1 to 10.
- For each number, print the number and its square on the same line, clearly labeled.
- Change the loop bound to 20 and confirm the output still updates correctly.

Program Link: https://www.jdoodle.com/ia/pb9a-skbp-geku

Source Code:

```
public class Week4_Basic_NumberSquarePrinter{
  public static void main(String[] args) {
    /* Use a for loop to count from 1 to 10 */
    /* Change the loop bound to 20 and confirm the output still updates correctly.*/
    int limit = 20;

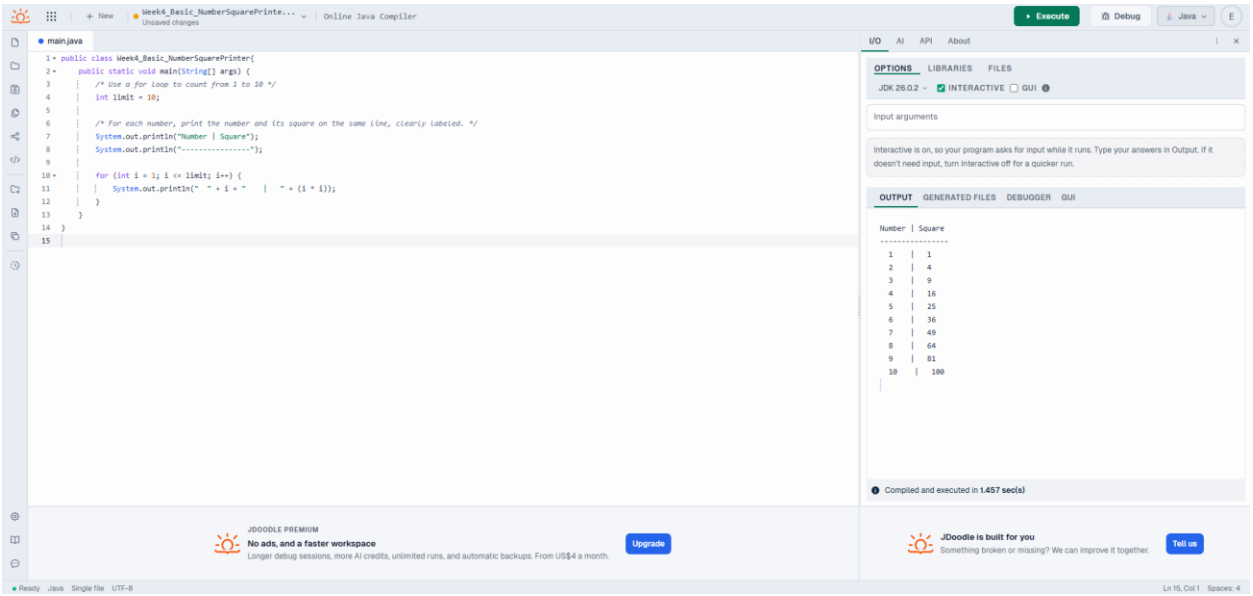
    /* For each number, print the number and its square on the same line, clearly labeled. */
    System.out.println("Number | Square");
    System.out.println("----------------");

    for (int i = 1; i <= limit; i++) {
      System.out.println("  " + i + "    |   " + (i * i));
    }
  }
}
```

Screenshots:

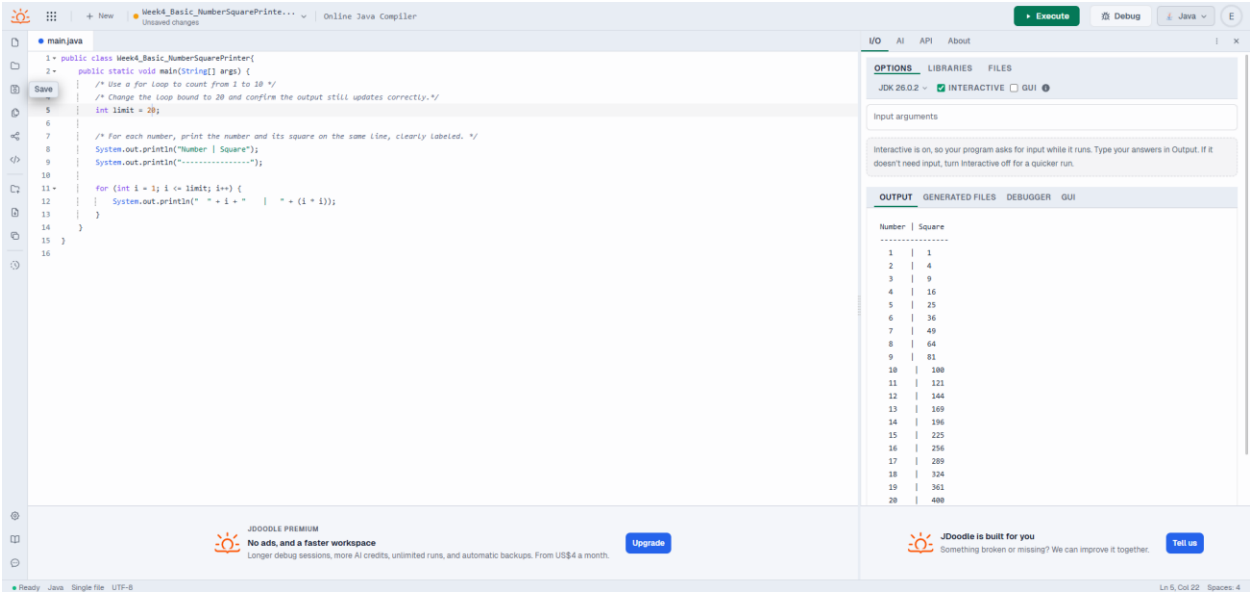
Result 1: up to 10

Program Link: https://www.jdoodle.com/ia/chcq-t5am-mvqd



Result 2: up to 20

Program Link: https://www.jdoodle.com/ia/pb9a-skbp-geku



Compiler Used:

https://www.jdoodle.com/online-java-compiler

INTERMEDIATE

Sum Until Sentinel Value

Use a while loop to keep accumulating input until a special stop value is entered.

Requirements:

- Use a while loop that keeps reading integers and adding them to a running total.
- Stop reading the instant the value -1 is entered, without including it in the total.
- Print the final total once the loop ends.
- Test with an input sequence that includes at least one negative number OTHER than -1.

Program Link: https://www.jdoodle.com/ia/vjdc-gzxj-bacv

Source Code:

```
import java.util.Scanner;

public class Week4_Intermediate_SumUntilSentinelValue {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    int total = 0;

    System.out.println("Enter numbers to add to the total.");
    System.out.println("Enter -1 to stop.");

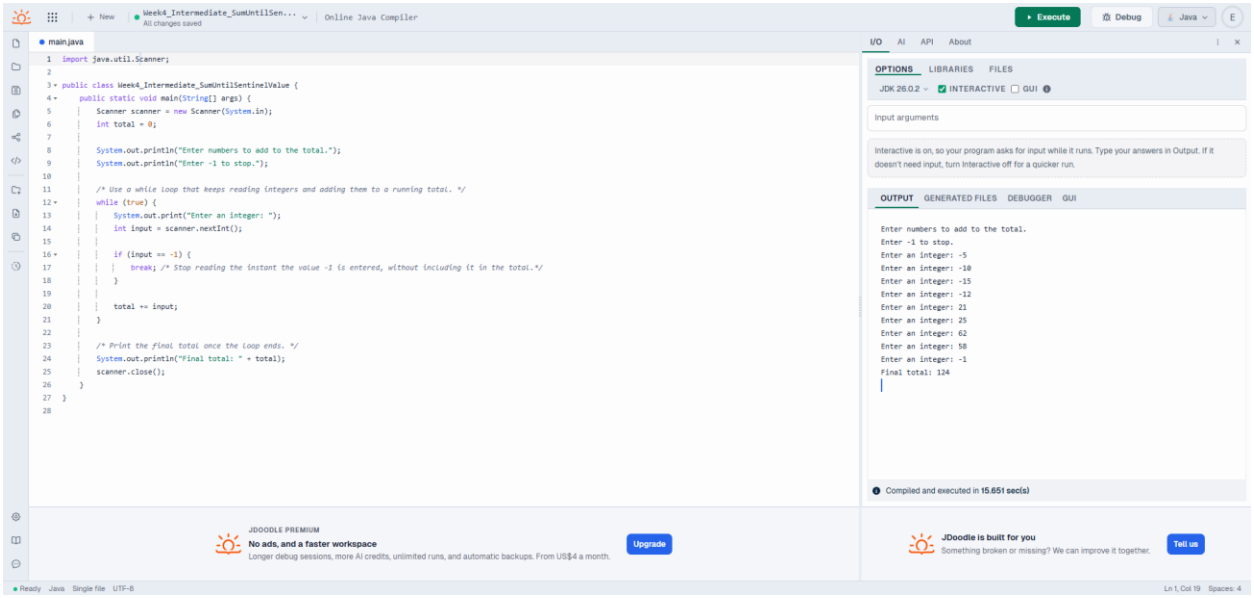
    /* Use a while loop that keeps reading integers and adding them to a running total. */
    while (true) {
      System.out.print("Enter an integer: ");
      int input = scanner.nextInt();

      if (input == -1) {
        break; /* Stop reading the instant the value -1 is entered, without including it in the total.*/
      }

      total += input;
    }

    /* Print the final total once the loop ends. */
    System.out.println("Final total: " + total);
    scanner.close();
  }
}
```

Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

ADVANCED

Pattern Generator With Early Exit

Use nested loops to print a pattern, incorporating break to stop under a special condition.

Requirements:

- Use nested loops to print a right-triangle or square pattern of a given size.
- Add a condition that uses break to stop the INNER loop early once a specific value or position is reached.
- Add a separate condition using continue to skip printing one specific row or column value.
- Clearly comment which lines demonstrate break and which demonstrate continue.

Program Link: https://www.jdoodle.com/ia/d8gs-puka-j33b

Source Code:

```
public class Week4_Advanced_PatternGeneratorwithEarlyExit {
  public static void main(String[] args) {
    int size = 5;
  /* Use nested loops to print a right-triangle or square pattern of a given size. */
    for (int row = 1; row <= size; row++) {

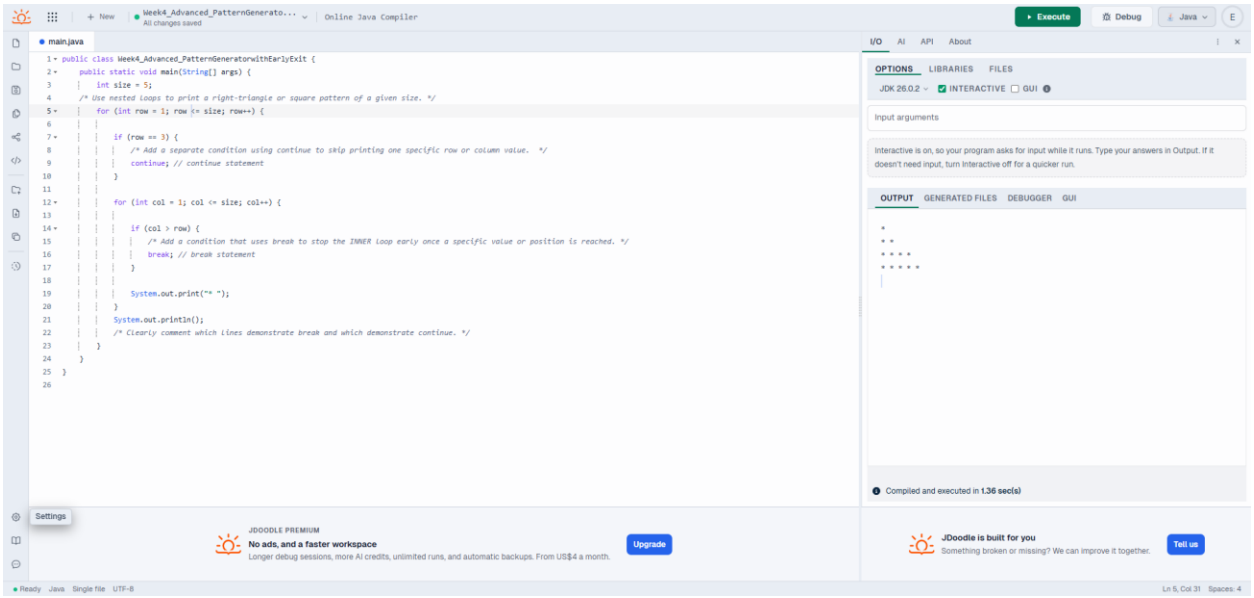
      if (row == 3) {
        /* Add a separate condition using continue to skip printing one specific row or column value.  */
        continue; // continue statement
      }

      for (int col = 1; col <= size; col++) {

        if (col > row) {
          /* Add a condition that uses break to stop the INNER loop early once a specific value or position is reached. */
          break; // break statement
        }

        System.out.print("* ");
      }
      System.out.println();
      /* Clearly comment which lines demonstrate break and which demonstrate continue. */
    }
  }
}
```

Screenshot:



Compiler Used:

https://www.jdoodle.com/online-java-compiler

EXPERT

Warehouse Inventory Scanner

Build a complete inventory system combining every looping and jump statement from this week.

Requirements:

- Represent a warehouse as a 2D array of item counts, using -1 to mean a damaged shelf.
- Use a do-while driven menu that runs until the user chooses to exit.
- Use a for loop with continue to total inventory while skipping damaged shelves.
- Use nested loops with break to find the first critically low shelf, and a while loop plus return to simulate restocking with input validation.

Program Link: https://www.jdoodle.com/ia/u9st-h8g2-7t5j

Source Code:

```
import java.util.Scanner;

public class Week4_Expert_WarehouseInventoryScanner {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    /*Represent a warehouse as a 2D array of item counts, using -1 to mean a damaged shelf.*/
    int[][] warehouse = {
      {20, 15, -1, 50},
      {10, -1, 3, 25}, // 3 is critically low
      {-1, 12, 8, 40}
    };

    int choice;

    /*Use a do-while driven menu that runs until the user chooses to exit.*/
    do {
      System.out.println("\n--- Warehouse Inventory System ---");
      System.out.println("1. Calculate Total Inventory");
      System.out.println("2. Restock First Critically Low Shelf");
      System.out.println("3. Exit");
      System.out.print("Select an option: ");
      choice = scanner.nextInt();

      if (choice == 1) {
        calculateTotal(warehouse);
      } else if (choice == 2) {
        restockShelf(warehouse, scanner);
      }

    } while (choice != 3);
```

```
    System.out.println("System exited.");
    scanner.close();
}

public static void calculateTotal(int[][] warehouse) {
    int total = 0;

    for (int i = 0; i < warehouse.length; i++) {
        for (int j = 0; j < warehouse[i].length; j++) {
            // for loop with continue to skip damaged shelves
            if (warehouse[i][j] == -1) {
                continue;
            }
            total += warehouse[i][j];
        }
    }
    System.out.println("Total available inventory: " + total);
}
/*Use a for loop with continue to total inventory while skipping damaged shelves.*/
/*•         Use nested loops with break to find the first critically low shelf,
and a while loop plus return to simulate restocking with input validation.*/

public static void restockShelf(int[][] warehouse, Scanner scanner) {
    int targetRow = -1;
    int targetCol = -1;

    // Nested loops with break to find the first critically low shelf (< 5 items)
    for (int i = 0; i < warehouse.length; i++) {
        for (int j = 0; j < warehouse[i].length; j++) {
            if (warehouse[i][j] != -1 && warehouse[i][j] < 5) {
                targetRow = i;
                targetCol = j;
                break; // Break inner loop
            }
        }
        if (targetRow != -1) {
            break; // Break outer loop once found
        }
    }

    if (targetRow == -1) {
        System.out.println("No critically low shelves found.");
        return;
    }

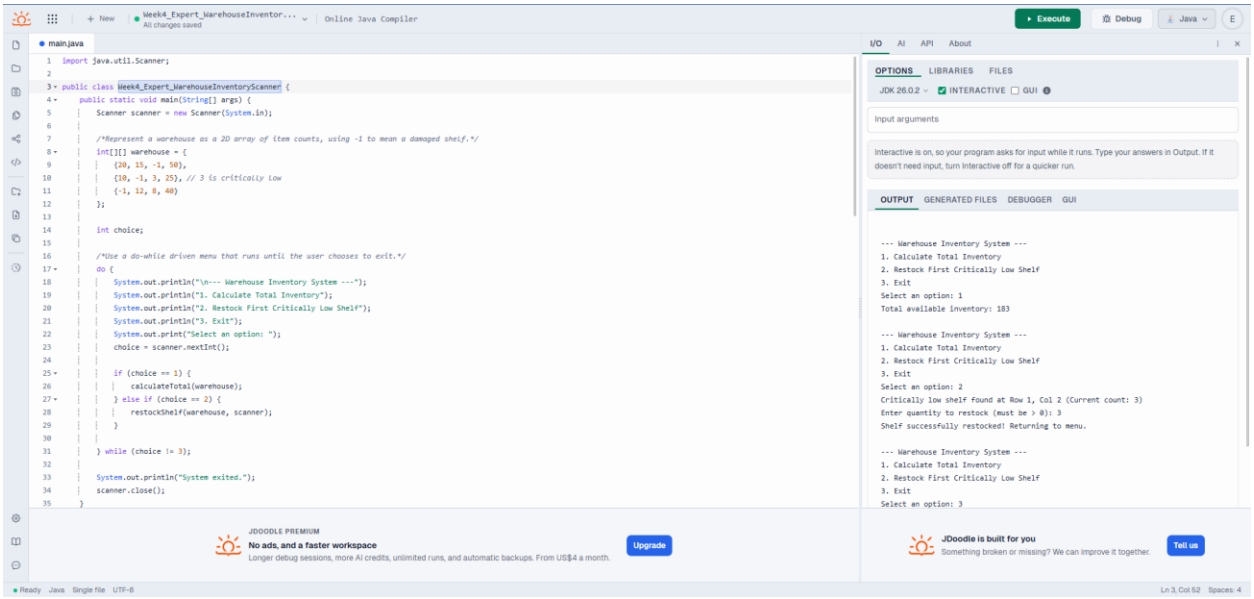
    System.out.println("Critically low shelf found at Row " + targetRow +
                       ", Col " + targetCol + " (Current count: " + warehouse[targetRow][targetCol] + ")");

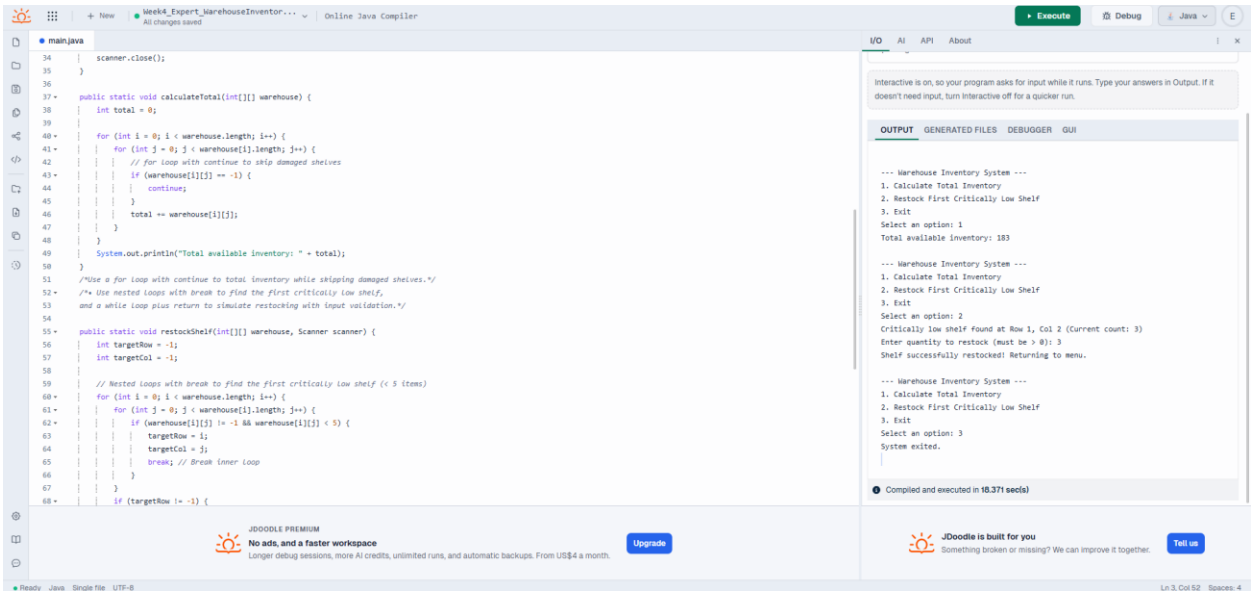
    // while loop plus return to simulate restocking with input validation
    while (true) {
        System.out.print("Enter quantity to restock (must be > 0): ");
        int amount = scanner.nextInt();

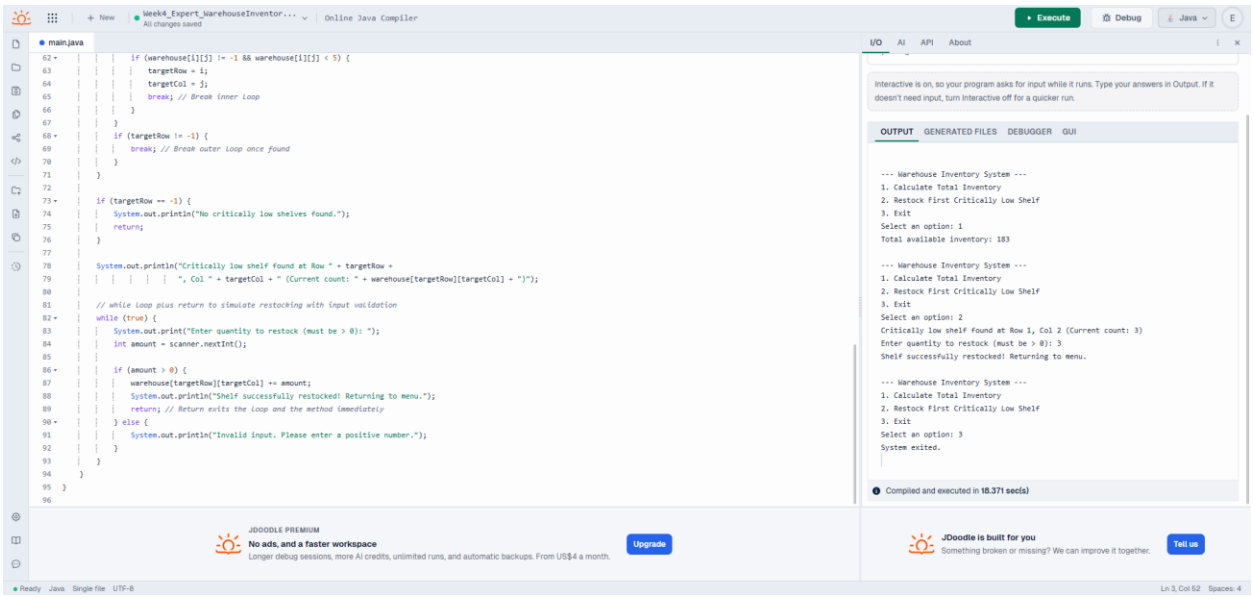
        if (amount > 0) {
            warehouse[targetRow][targetCol] += amount;
            System.out.println("Shelf successfully restocked! Returning to menu.");
```

```
            return; // Return exits the loop and the method immediately
        } else {
            System.out.println("Invalid input. Please enter a positive number.");
        }
    }
  }
}
```

Screenshots:







Compiler Used:

https://www.jdoodle.com/online-java-compiler